

proyecto credits



Proyecto: Credit Risk Scoring para Fintech

Categoría: Modelos Predictivos de Negocio **Skills cubiertos:** Clasificación + Regresión + SQL + FastAPI + Power BI + CI/CD **Nivel:** Intermedio-Avanzado **Tiempo estimado:** 3-5 semanas

1. PROBLEMA DE NEGOCIO

¿Qué resuelve?

Una fintech o banco digital necesita decidir en segundos si aprueba o rechaza una solicitud de crédito, y en qué monto. Hacerlo mal en cualquier dirección cuesta dinero: aprobar a quien no pagará genera cartera vencida; rechazar a quien sí pagaría pierde ingreso por intereses y clientes.

Este proyecto construye un **sistema de score crediticio** que:

- **Clasifica** si un solicitante incurrirá en incumplimiento de pago (**TARGET = 1/0**)
- **Regresiona** el monto de pérdida esperada si el cliente defaultea (Loss Given Default)
- Los dos juntos calculan el **Expected Loss = P(Default) × Pérdida Estimada**

¿Por qué es valioso para una empresa real?

- El sector fintech en México crece >25% anual (Kueski, Konfío, Credijusto, etc.)
- Los bancos tradicionales (BBVA, Banamex, Santander) tienen equipos de riesgo crediticio
- Un modelo bien documentado + API desplegada demuestra que puedes productivizar ML
- Es uno de los casos de uso más demandados en ofertas de Junior DS en LATAM

2. DATASET

Principal: Home Credit Default Risk

- **Fuente:** <https://www.kaggle.com/c/home-credit-default-risk/data>
- **Tamaño:** ~307,000 solicitudes de crédito, 120+ features
- **Tablas disponibles:**

Archivo	Contenido	Relación
application_train.csv	Solicitudes con target	tabla principal
bureau.csv	Historial crediticio externo	muchas filas por cliente
previous_application.csv	Solicitudes anteriores	muchas filas por cliente
installments_payments.csv	Pagos de cuotas	muchas filas por cliente

- **Target:** TARGET (0 = pagó, 1 = incumplió)

Estrategia: empieza solo con application_train.csv + bureau.csv . Agrega las otras tablas cuando el pipeline base ya funcione.

Alternativa más ligera (para iterar rápido primero)

- **Give Me Some Credit** — <https://www.kaggle.com/c/GiveMeSomeCredit>
- ~150,000 filas, 10 features, una sola tabla
- Ideal para validar el pipeline completo antes de escalar al dataset principal

Por qué bureau.csv requiere tratamiento especial

application_train.csv tiene una fila por cliente. bureau.csv tiene muchas filas por cliente (una por cada crédito previo).

No puedes hacer JOIN directo — explotan los registros. El script 02_feature_bureau.sql colapsa esas muchas filas en una sola por cliente mediante agregaciones (COUNT, MAX, AVG) antes del JOIN.

3. ARQUITECTURA DEL PROYECTO

Separación de entornos

1 DESARROLLO (tu máquina)

PRODUCCIÓN (tu VPS)

<pre> 2 3 Makefile orquesta el pipeline 4 uv + .venv para dependencias 5 scripts/ cargan y transforman 6 notebooks/ documentan decisiones 7 sql/ define las transformaciones 8 models/ guarda model.pkl </pre>	<pre> docker-compose.yml levanta: └─ PostgreSQL └─ FastAPI └─ Nginx (HTTPS) </pre>
--	--

El EDA, la limpieza y el entrenamiento son **exclusivamente de desarrollo**. En producción solo existe el modelo serializado (`model.pkl`) y la API que lo sirve.

Flujo de datos completo

```

1 CSV de Kaggle (local)
2   |
3   | scripts/load_raw.py – único script que toca los CSVs
4   ▼
5 PostgreSQL (Docker) — schema: raw
6   | raw.applications ← copia exacta del CSV, nunca se
toca
7   | raw.bureau
8   |
9   | sql/01_staging.sql – limpieza básica en SQL
10  ▼
11 PostgreSQL — schema: staging
12   | staging.stg_applications ← tipos corregidos, nulos
imputados
13   | staging.stg_bureau_features ← bureau colapsado a 1 fila por
cliente
14   |
15   | ★ notebooks/01_eda.ipynb – EDA VA AQUÍ, no al final
16   | Python lee staging.*, explora, decide qué features construir
17   |
18   | sql/02_features.sql – JOIN + features derivados
19   ▼
20 PostgreSQL — schema: processed
21   | processed.vw_features_modelo ← VIEW lista para scikit-learn
22   |
23   | pd.read_sql("SELECT * FROM processed.vw_features_modelo",
engine)
24   ▼
25 DataFrame → scikit-learn → models/model.pkl
26   |
27   ▼

```

¿Por qué SQL hace la limpieza y no Python?

SQL hace	Python hace
Renombrar columnas	Imputación con KNN o modelo
Corregir tipos (<code>CAST</code>)	Detección de outliers con IQR
Imputar con valor fijo (<code>COALESCE</code>)	Encoding (One-Hot, WoE)
Filtrar filas inútiles	Normalización / escalado
JOINS entre tablas	Visualizaciones del EDA
Features simples (<code>income / loan</code>)	Features que requieren librerías

Regla práctica: si lo puedes escribir en un `SELECT`, hazlo en SQL. Si necesitas una librería de Python o estadística avanzada, hazlo en Python.

¿Por qué los schemas no se sobrescriben?

```

1  raw.*           → fuente de verdad, nunca cambia
2                  "así llegó la data del mundo real"
3
4  staging.*       → decisiones de limpieza documentadas
5                  "esto fue lo que decidimos corregir y por qué"
6
7  processed.*    → features listos para el modelo
8                  "esto es lo que el modelo ve"
```

En los notebooks puedes comparar los tres estados y mostrar exactamente qué decisiones tomaste — eso es el storytelling de datos que diferencia tu portfolio.

4. STACK TÉCNICO

Gestión de dependencias — uv

Un solo ambiente virtual en la raíz del proyecto, con grupos separados:

```

1  # pyproject.toml
2  [project]
```

```

3 dependencies = [
4     "pandas", "numpy", "scikit-learn",
5     "sqlalchemy", "psycopg2-binary", "joblib",
6 ]
7
8 [dependency-groups]
9 dev = ["jupyter", "matplotlib", "seaborn", "plotly", "ipykernel"]
10 api = ["fastapi", "uvicorn", "pydantic", "alembic"]

```

```

1 uv sync --all-groups    # desarrollo local (todo)
2 uv sync --group api     # solo para el Dockerfile

```

Docker — solo servicios continuos

```

1 # docker-compose.yml (raíz del proyecto)
2 services:
3
4     db:
5         image: postgres:15
6         environment:
7             POSTGRES_DB: credit_risk
8             POSTGRES_USER: ${DB_USER}
9             POSTGRES_PASSWORD: ${DB_PASS}
10        ports:
11            - "5432:5432"          # accesible desde tu máquina en
localhost:5432
12        volumes:
13            - postgres_data:/var/lib/postgresql/data    # named volume, no bind
mount
14            - ./db/init:/docker-entrypoint-initdb.d    # 00_schemas.sql corre al
arrancar
15        healthcheck:
16            test: ["CMD", "pg_isready", "-U", "${DB_USER}", "-d", "credit_risk"]
17            interval: 5s
18            retries: 5
19
20        api:
21            build: ./api
22            depends_on:
23                db:
24                    condition: service_healthy    # espera a que pg_isready pase
25            ports:
26                - "8000:8000"
27            environment:

```

```

28     DATABASE_URL: postgresql://${DB_USER}:${DB_PASS}@db:5432/credit_risk
29     #                                     ↑
30     #                                     nombre del servicio Docker, no localhost
31     volumes:
32     - ./models:/app/models    # comparte model.pkl con el contenedor
33
34     volumes:
35     postgres_data:    # Docker administra dónde vive esto, fuera del proyecto

```

Importante: Los notebooks, scripts y el pipeline de datos **no van en Docker**. Solo van los servicios que corren continuamente en producción.

Conexiones según el entorno

```

1  # scripts/ y notebooks/ – corren en tu máquina
2  DATABASE_URL = "postgresql://admin:secret@localhost:5432/credit_risk"
3  #                                     ↑ puerto expuesto por Docker al
   host
4
5  # api/main.py – corre dentro de Docker
6  DATABASE_URL = "postgresql://admin:secret@db:5432/credit_risk"
7  #                                     ↑ nombre del servicio en red
   Docker

```

Ambas URLs vienen de `.env` — nunca hardcodeadas en el código.

Makefile — orquesta el pipeline de desarrollo

```

1  include .env
2  export
3
4  PYTHON = uv run python
5  DB_URL = postgresql://$(DB_USER):$(DB_PASS)@localhost:5432/credit_risk
6
7  .PHONY: all setup load staging features train clean
8
9  # Corre el pipeline completo de una vez
10 all: setup load staging features
11     @echo "✅ Pipeline completo. Abre los notebooks."
12
13 setup:
14     @echo "🐳 Levantando PostgreSQL..."
15     @docker-compose up -d db

```

```

16     @until pg_isready -h localhost -p 5432 -U $(DB_USER); do \
17         echo "Esperando PostgreSQL..."; \
18         sleep 2; \
19     done
20
21 load: setup
22     @echo "📁 Cargando CSVs a raw.*..."
23     @$(PYTHON) scripts/load_raw.py
24
25 staging: load
26     @echo "🧹 Limpieza SQL → staging.*..."
27     @psql $(DB_URL) -f sql/01_staging.sql
28
29 features: staging
30     @echo "⚙️ Features → processed.*..."
31     @psql $(DB_URL) -f sql/02_features.sql
32
33 train: features
34     @echo "🧠 Entrenando modelo..."
35     @$(PYTHON) scripts/train.py
36
37 deploy:
38     @docker-compose build api
39     @docker-compose up -d api
40
41 clean:
42     @docker-compose down -v # borra contenedores y volúmenes

```

```

1  make all      # pipeline completo
2  make train    # asume que features ya existen
3  make deploy   # construye y sube la API
4  make clean    # borra todo y empieza de cero
5  make -n all   # dry run: muestra qué correría sin ejecutar

```

5. PLAN DE ANÁLISIS

Este es el mapa de trabajo, no las respuestas. El análisis es tuyo.

Fase 0 — Setup del proyecto

- ☐ Crear repo en GitHub con la estructura de carpetas definida
- ☐ Configurar `uv`, `pyproject.toml` y grupos de dependencias

- ☐ Configurar `docker-compose.yml` con PostgreSQL y named volume
- ☐ Crear `.env` local (basado en `.env.example`) con credenciales
- ☐ Correr `make setup` y verificar que `pg_isready` pasa
- ☐ Leer el diccionario de variables del dataset de Kaggle

Fase 1 — Carga de datos raw

- ☐ Escribir `scripts/load_raw.py` — carga CSVs a `raw.*` sin transformar nada
- ☐ Verificar en psql que los registros cargaron correctamente
- ☐ Documentar en `data/README.md` cómo descargar los CSV de Kaggle

Fase 2 — Staging SQL (`sql/01_staging.sql`)

- ☐ Corregir tipos de dato (`CAST`)
- ☐ Renombrar columnas a nombres interpretables
- ☐ Imputar nulos obvios con `COALESCE`
- ☐ Filtrar filas inútiles
- ☐ Crear `staging.stg_bureau_features` colapsando `raw.bureau` a 1 fila por cliente

Fase 3 — EDA (`notebooks/01_eda.ipynb`)

- ☐ Python lee `staging.stg_applications` — no CSV, no raw
- ☐ Comparar `raw.*` vs `staging.*` para mostrar el efecto de la limpieza
- ☐ Análisis de calidad: nulos restantes, tipos, duplicados
- ☐ Distribución del target — estrategia ante el desbalance de clases (~8%)
- ☐ Análisis bivariado: relación de cada variable con `TARGET`
- ☐ Detección de outliers en variables financieras
- ☐ Mínimo 6–8 visualizaciones con conclusión de negocio bajo cada una
- ☐ Resumen ejecutivo final: "Qué aprendí en 5 bullets" → esto dicta qué va en el SQL siguiente

Fase 4 — Features SQL (`sql/02_features.sql`)

- ☐ JOIN entre `stg_applications` y `stg_bureau_features`
- ☐ Construir features derivados: ratios, agregaciones, flags
- ☐ Crear `processed.vw_features_modelo` como VIEW final
- ☐ Verificar que la vista tiene exactamente 1 fila por cliente

Fase 5 — Preparación para modelado

(`src/features/engineering.py`)

- ☐ Python lee `processed.vw_features_modelo`
- ☐ Encoding de categóricas (evalúa WoE para variables crediticias)
- ☐ Feature selection: correlación, importancia preliminar, VIF
- ☐ División train/validation/test estratificada
- ☐ Guardar sklearn Pipeline con joblib — este mismo objeto lo usará la API

Fase 6 — Clasificación (`src/models/train_classifier.py`)

- ☐ Baseline: Regresión Logística — interpreta coeficientes en lenguaje de negocio
- ☐ Intermedio: KNN, Naive Bayes con LDA/QDA — documenta por qué sirven o no
- ☐ Avanzado: Random Forest o LightGBM
- ☐ Cross-validation estratificada (mínimo 5-fold)
- ☐ Comparar en tabla: AUC-ROC, KS, Precision, Recall, F1
- ☐ Seleccionar umbral con criterio de negocio (no el default 0.5)
- ☐ Documentar decisiones en `notebooks/02_clasificacion.ipynb`

Fase 7 — Regresión auxiliar

(`src/models/train_regressor.py`)

- ☐ Filtrar solo clientes que defaultearon
- ☐ Modelar monto de pérdida: Lineal, Ridge, Lasso
- ☐ Calcular Expected Loss = $P(\text{Default}) \times \text{Loss Given Default}$
- ☐ Interpretar en dinero: "Para este perfil, la pérdida esperada es X pesos"
- ☐ Documentar en `notebooks/03_regresion.ipynb`

Fase 8 — Monitoreo (`notebooks/04_monitoreo.ipynb`)

- ☐ Simular data drift: ¿qué pasa con el score si cambia la distribución de ingresos?
- ☐ Calcular PSI (Population Stability Index) entre train y muestra "nueva"
- ☐ Definir umbrales de alerta: $\text{PSI} > 0.2 \rightarrow$ reentrenar
- ☐ Documentar protocolo de reentrenamiento

Fase 9 — API (`api/`)

- [] `POST /predict` — recibe variables del solicitante, retorna score + decisión + pérdida esperada
- [] `GET /health` — monitoreo del servicio
- Validación de inputs con Pydantic
- Reutiliza `src/features/engineering.py` — sin duplicar código
- Alembic para gestionar tabla `api.predictions` (log de predicciones)
- Dockerfile con `uv sync --group api --no-dev`

6. MÉTRICAS DE ÉXITO

Métrica	Qué mide	Interpretación de negocio
AUC-ROC	Capacidad de separar defaulters	>0.75 útil; >0.80 muy bueno
KS Statistic	Separación máxima entre distribuciones	Estándar financiero; >30 es benchmark
Precision @ umbral	De los rechazados, ¿cuántos realmente hubieran defaultado?	Costo de rechazar clientes buenos
Recall @ umbral	De todos los defaulters, ¿cuántos detectamos?	Costo de aprobar clientes malos
RMSE (regresión)	Error en estimación de pérdida	Interpretable en pesos
PSI	Estabilidad del score en el tiempo	<0.1 estable; >0.2 reentrenar

Diferenciador de portfolio: monetiza la matriz de confusión. En vez de "Falsos Negativos = 120", calcula cuánto dinero representan esos 120 clientes mal clasificados. Eso es lo que entiende un hiring manager.

7. ESTRUCTURA DEL REPO

```

1  credit-risk-scoring/
2  |
3  ├── .env.example           ← plantilla de variables (SÍ al repo)
4  ├── .gitignore
5  ├── docker-compose.yml     ← raíz del proyecto, orquesta DB + API
6  ├── Makefile               ← orquesta el pipeline de desarrollo
7  ├── pyproject.toml         ← dependencias con grupos (uv)

```

```

8 |— uv.lock                                ← lockfile exacto (Sí al repo)
9 |
10 |— data/
11 |   |— README.md                        ← instrucciones para descargar los CSV de
Kaggle
12 |                                       (los CSV van en .gitignore)
13 |
14 |— db/
15 |   |— init/
16 |       |— 00_schemas.sql             ← corre automático al arrancar PostgreSQL
CREATE SCHEMA raw, staging, processed
17 |
18 |
19 |— sql/
20 |   |— 01_staging.sql                  ← limpieza: tipos, nulos, renombres
21 |   |— 02_features.sql                ← JOINS + features derivados → VIEW final
22 |
23 |— scripts/
24 |   |— load_raw.py                    ← único script que toca los CSVs
25 |   |— pipeline.py                   ← alternativa al Makefile, verifica qué ya
corrió
26 |
27 |— src/
28 |   |— data/
29 |       |— load.py                    ← conexión a DB, pd.read_sql()
30 |   |— features/
31 |       |— engineering.py             ← encoding, scaling, sklearn Pipeline
32 |   |— models/
33 |       |— train_classifier.py
34 |       |— train_regressor.py
35 |       |— evaluate.py                ← métricas + monetización de la confusion
matrix
36 |
37 |— notebooks/
38 |   |— 01_eda.ipynb                   ← importa src/, narrativa + visualizaciones
39 |   |— 02_clasificacion.ipynb
40 |   |— 03_regresion.ipynb
41 |   |— 04_monitoreo.ipynb
42 |
43 |— api/
44 |   |— main.py                        ← FastAPI, importa src/ sin duplicar código
45 |   |— schemas.py                     ← Pydantic models
46 |   |— migrations/                   ← Alembic (solo para tablas de la API)
47 |   |— Dockerfile
48 |
49 |— models/
50 |   |— model_v1.pkl                   ← artefacto entrenado (si <50MB va al repo)

```

```

51 |
52 | └─ tests/
53 |   └─ test_features.py      ← prueba src/features/engineering.py
54 |   └─ test_api.py          ← prueba endpoints de FastAPI
55 |
56 | └─ dashboard/
57 |   └─ credit_risk.pbix
58 |   └─ credit_risk.pdf      ← export para el repo
59 |
60 | └─ .github/
61 |   └─ workflows/
62 |     └─ deploy.yml         ← CI/CD: test → build → deploy al VPS

```

Lo que va al repo vs lo que no

1	SÍ al repo	NO al repo
2	_____	_____
3	sql/	data/raw/*.csv (pesados)
4	scripts/	.venv/
5	src/	.env (credenciales)
6	notebooks/	data/postgres/ (binarios de DB)
7	pyproject.toml	__pycache__/
8	uv.lock	.ipynb_checkpoints/
9	docker-compose.yml	
10	Makefile	
11	.env.example	
12	models/model_v1.pkl (si <50MB)	

README debe incluir

- Badge de CI/CD (GitHub Actions pasando)
- Descripción del problema en 2-3 oraciones
- Diagrama de arquitectura (Mermaid o imagen)
- Tabla de resultados del modelo (AUC, KS, RMSE)
- Cómo reproducir el proyecto: `git clone` → `make all` → `jupyter notebook`
- Link a la API live con ejemplo de uso en `curl`
- Screenshot del dashboard de Power BI

8. NIVEL DE DIFICULTAD

Intermedio-Avanzado

Componente	Dificultad
Setup Docker + PostgreSQL + uv	☆☆ Intermedio
Pipeline SQL (raw → staging → processed)	☆☆ Intermedio
EDA con storytelling de negocio	☆☆ Intermedio
Clasificación + selección de umbral	☆☆ Intermedio
Regresión + Expected Loss monetizado	☆☆☆ Avanzado
Monitoreo con PSI	☆☆☆ Avanzado
FastAPI + Pydantic + Alembic	☆☆ Intermedio
CI/CD con GitHub Actions	☆☆ Intermedio

Tiempo estimado:

- Semana 1: Setup + SQL pipeline + EDA
- Semana 2: Features + Clasificación
- Semana 3: Regresión + Monitoreo
- Semana 4–5: API + Power BI + README + CI/CD

¿Por qué este proyecto primero?

1. **Máxima demanda en México:** BBVA, Santander, Banorte, Kueski, Konfío
— todos tienen equipos de riesgo crediticio buscando perfiles como el tuyo
2. **Combina lo que ya sabes:** regresión logística, Ridge, Lasso, KNN, Bayes
— todos tienen justificación natural en riesgo crediticio
3. **Diferenciador real:** la mayoría de portfolios termina en el modelo. Tú llegas al monitoreo, al despliegue y a la monetización de métricas
4. **El repo es reproducible:** cualquier reviewer puede clonar y correr `make all` para ver el pipeline completo desde cero

Primer comando cuando tengas el repo creado: `make setup` para levantar PostgreSQL y verificar que `pg_isready` pasa.